

## Projet CI/CD Sécurisé avec Docker, GitLab et Harbor

### Sujet

« Sécurisation de la Supply Chain Logicielle: mise en place d'un pipeline CI/CD sécurisé avec GitLab, Docker et Harbor »

### Objectif global

Concevoir, implémenter et démontrer une chaîne CI/CD sécurisée assurant la construction, la vérification, la signature, le scan et le déploiement contrôlé d'images Docker via GitLab CI/CD et Harbor. Ce projet s'inscrit dans le cadre du DevSecOps et vise à garantir la confiance et l'intégrité du code et des artefacts tout au long du cycle de vie logiciel.

### Contexte

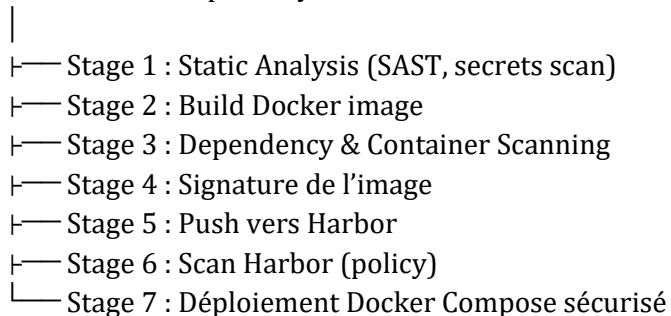
Les attaques sur la supply chain logicielle (ex. SolarWinds, Log4j, Docker Hub poisoning) ont montré la nécessité de sécuriser les pipelines CI/CD. Harbor permet non seulement d'héberger des images Docker, mais aussi de scanner les vulnérabilités, gérer les accès (RBAC) et signer les images. Le projet simule une organisation DevSecOps sécurisant son pipeline end-to-end avec GitLab CI/CD.

### Objectifs techniques détaillés

- Créer un pipeline GitLab CI/CD sécurisé incluant build, SAST, scans, signature et déploiement.
- Mettre en place la gestion sécurisée des secrets et des politiques de sécurité Harbor.
- Réaliser une analyse de risques du pipeline (STRIDE) et vérifier la conformité OWASP.

### Architecture du projet

Développeur → GitLab Repository



Harbor ← (registre privé + scan vulnérabilités + signature)  
Serveur distant (Docker Compose, monitoring sécurité)

## Stack technique

- GitLab CE / GitLab.com – CI/CD sécurisé
- Harbor – Registre privé, scan vulnérabilités, signature
- Docker / Docker Compose – Conteneurisation et déploiement
- Trivy / Clair – Scan images
- Cosign / Notary v2 – Signature et vérification d'images
- Bandit / Semgrep – Scan de code source (SAST)
- Gitleaks – Détection de secrets dans le code
- OWASP Dependency Check – Scan des dépendances
- Python 3.11 + FastAPI – Langage et application déployée
- Prometheus + Grafana – Monitoring sécurité (optionnel)

## Plan de travail (3 mois / 12 semaines)

### Phase 1 — Semaine 1 à 3 : Conception et setup

Analyse des menaces, spécifications, installation GitLab, Harbor et Docker Compose.

### Phase 2 — Semaine 4 à 6 : Intégration CI/CD

Création du pipeline GitLab CI/CD (SAST, build, scan, push Harbor).

### Phase 3 — Semaine 7 à 9 : Sécurisation et conformité

Signature des images, politiques Harbor, gestion des secrets, déploiement Compose.

### Phase 4 — Semaine 10 à 12 : Audit & documentation

Audit du pipeline, monitoring sécurité, rapport final et démonstration.

## Structure du projet

```
.gitlab-ci.yml
README.md
/docs/
├── cahier_des_charges.md
├── analyse_risques.md
├── rapport_final.pdf
/src/
├── app.py
└── utils.py
/tests/
└── test_app.py
/docker/
└── Dockerfile
/docker-compose.yml
/scripts/
└── scan.sh
```

Énoncé par M. Bonitah RAMBELOSON  
Consultant DevOps | Cloud Engineer | MLOps Practitioner

└─ sign.sh  
└─ verify.sh

### Critères d'évaluation (proposition)

| Critère                                         | Pondération |
|-------------------------------------------------|-------------|
| Pipeline CI/CD complet et automatisé            | 30%         |
| Sécurisation des étapes (SAST, scan, signature) | 25%         |
| Gestion des secrets et conformité OWASP         | 20%         |
| Déploiement automatisé sécurisé (Compose)       | 15%         |
| Rapport et démonstration technique              | 10%         |

### Axes d'approfondissement (pour excellence)

- Intégrer Vault pour la gestion dynamique des secrets.
- Mettre en place Policy-as-Code (OPA / Conftest).
- Générer et vérifier un SBOM (Syft).
- Simuler une attaque supply chain et prouver la résilience du pipeline.
- Monitoring sécurité via Grafana (métriques vulnérabilités dans le temps).